

Cours 2

Programmation orientée objet

Jérémy Kairis

[https://docs.microsoft.com/fr-be/
training/paths/csharp-first-steps/](https://docs.microsoft.com/fr-be/training/paths/csharp-first-steps/)

Les conversions

Les conversions

De type valeur à *string*

Tout élément en C# possède une méthode “***ToString()***”, dans le cadre des types valeur, nous pourrions utiliser cette méthode pour transformer notre type valeur en type “***string***”.

```
int entier = 42;
```

```
string chaineEntier = entier.ToString();
```

Les conversions

De *string* à un type valeur

Pour l'inverse, c'est-à-dire de “***string***” vers le type valeur plusieurs solutions sont offertes :

- La classe “***Convert***”
- La méthode “***Parse***”
- La méthode “***TryParse***”

Chacune de ces méthodes a ses avantages et inconvénients, mais la plus sûre reste la méthode “***TryParse***”.

Les conversions

La classe Convert

La classe “**Convert**” propose différentes méthodes permettant de convertir vers un type donné.

Si elle ne parvient pas à faire la conversion, la méthode lèvera une erreur à l’exécution du programme,

De plus, elle permet la conversion de type vers type comme des “**double**” en “**int**”, des “**bool**” en “**double**”, etc.

```
double valeurDouble = 3.14;
```

```
int valeurInt = Convert.ToInt32(valeurDouble);
```

```
bool estVrai = true;
```

```
double valeurDoubleBool = Convert.ToDouble(estVrai);
```

```
string chaineNonValide = "123abc";
```

```
int valeurIntNonValide = Convert.ToInt32(chaineNonValide);
```


Les conversions

La méthode *Parse*

Tout type valeur possède les méthodes “***Parse***” et “***TryParse***”, ces méthodes ne permettent que de lire une chaîne de caractères et de la convertir en fonction du type valeur choisi.

La méthode “***Parse***” reçoit en paramètre une “***string***” et tente de convertir cette chaîne dans le type spécifié.

Si elle n’y parvient pas, une erreur sera déclenchée à l’exécution.

```
string chaineIntValide = "42";
```

```
int valeurIntParse = int.Parse(chaineIntValide);
```

Les conversions

La méthode *TryParse*

La méthode “***TryParse***” quant à elle va recevoir en paramètre une “***string***” et un” second paramètre du même type que celui du résultat de la conversion précédé du mot-clé “***out***”.

Si elle parvient à convertir, la méthode retournera “***true***” et la valeur sera stockée dans la 2ème variable passée en paramètre.

Si pas, la méthode retournera “***false***”, la valeur de “***default(T)***” sera assignée à la 2ème variable passée en paramètre et aucune erreur ne sera déclenchée à l’exécution.


```
string chaineIntNonValide = "abc";
```

```
int valeurIntTryParse;
```

```
bool ok = int.TryParse(chaineIntNonValide, out valeurIntTryParse);
```

Les conversions

Conversions implicites

En dehors des conversions de et vers des “***string***”, C# prend en charge différentes conversions automatiques, appelées conversions implicites.

Ces conversions implicites sont autorisées par le compilateur car celui-ci pourra garantir que ces conversions n’affecteront d’aucune manière leur valeur.

Exemples:

- De “***int***” vers “***long***”, “***float***”, “***double***”, “***decimal***”.
- De “***float***” vers “***double***”.

```
int entier = 42;
```

```
long longEntier = entier;
```

```
float valeurFlottante = entier;
```

```
double valeurDouble = entier;
```

```
decimal valeurDecimal = entier;
```

```
float valeurFlottante = 3.14;
```

```
double valeurDouble = valeurFlottante;
```


Les conversions

Conversions explicites

Dans les autres cas nous devons spécifier la conversion de manière explicite.

Pour ce faire, imaginons la situation suivante, on nous demande de copier des fichiers d'un répertoire à un autre et afficher le pourcentage effectué par rapport au traitement complet.

Notre solution consisterait à déclarer la valeur littéral "100" en "***float***".

Nous devons spécifier au compilateur que nous voulons transtyper notre "***float***" en "***int***" et que par conséquent nous prenons la responsabilité en cas de perte de données (précision).

```
Console.Write("Entrez votre nom : ");
```

```
string nom = Console.ReadLine();
```

```
Console.Write("Entrez votre âge : ");
```

```
int age = Convert.ToInt32(Console.ReadLine());
```

```
Console.WriteLine("Bonjour, " + nom + "! Vous avez " + age + " ans.");
```


Exercices

Conversions

1. Écrivez un programme qui demande à l'utilisateur de saisir un nombre entier, le multiplie par 2, convertit le résultat en chaîne de caractères, puis affiche la chaîne résultante.
2. Demandez à l'utilisateur de saisir un nombre décimal, puis convertissez-le en un nombre entier et affichez le résultat.
3. Écrivez un programme qui convertit une température en degrés Celsius en degrés Fahrenheit, puis affiche la température en utilisant une phrase descriptive.